

强化学习复习笔记

基于课程内容整理

目录

Part I

基础理论

1 强化学习概述

1.1 强化学习能解决的问题

强化学习适合解决序贯决策、复杂控制和交互优化问题，典型应用包括：

- 非线性控制、视频游戏、机器人控制（走路、站立）
- AlphaGo、多智能体协作（星际争霸）
- 海量智能体问题、可控核聚变控制
- 大语言模型中的人类反馈优化（如 ChatGPT 的 RLHF）

典型场景：可控核聚变

托卡马克装置利用电磁力约束核聚变时的超高温等离子体。关键点：

- 控制等离子体形状，服务于发电
- 热传导不能过多（容器融化）也不能过少（发电效率低）

典型场景：ChatGPT 训练思路

1. 收集示例数据，得到有监督微调策略 SFT
2. 收集人工标注的对比数据，训练回报模型
3. 使用 PPO 算法和学到的回报模型进一步优化 SFT

1.2 强化学习与监督学习的联系和区别

相同点：都从数据中学习；都会逐步改善性能。

不同点：

- 数据类型：监督学习需要标签数据；强化学习需要与环境交互得到的数据
- 优化目标：监督学习优化预测误差；强化学习优化长期累积回报

1.3 强化学习与其他优化方法

序贯最优决策的其他方法：

- 状态空间很小时：线性规划（资源分配、调度）
- 模型已知、回报函数解析时：动态规划或最优控制
- 控制策略较简单时：进化算法（遗传算法）

最优控制的一般形式

状态方程：

$$\dot{X} = f(t, X, U), \quad X(t_0) = X_0$$

性能指标函数：

$$J(x, t) = \phi(X(t_f), t_f) + \int_t^{t_f} L(x(\tau), u(\tau), \tau) d\tau$$

最优值函数：

$$V(X, t) = \min_{u \in \Omega} \left[\phi(X(t_f), t_f) + \int_t^{t_f} L(x(\tau), u(\tau), \tau) d\tau \right]$$

对应 HJB 方程：

$$-\frac{\partial V}{\partial t} = \min_{u \in U} \left\{ L(x, t, u) + \left(\frac{\partial V}{\partial x} \right)^T f(x, t, u) \right\}$$

关键点

当模型未知或非常复杂、回报函数未知时，强化学习通过采样和环境交互来学习最优策略。

1.4 基本概念

策略（Policy）

策略是从状态空间到动作空间的映射：

$$\pi(\cdot | s) : s \rightarrow a$$

奖赏（Reward）

环境在每个时间步给智能体的立即回报。

值函数（Value Function）

从某个状态开始，按照某个策略行动时所能获得的累积回报。

学习目标

使长期累积回报最大，即让值函数最大。

1.5 与环境交互的学习过程

强化学习通过与环境交互产生数据流，智能体利用这些数据学习并优化行为，通常放在马尔可夫决策过程（MDP）框架下理解。

强化学习的两个核心特征：

- 试错探索
- 延迟回报

井字游戏示例

- 设置值函数列表，每个值表示从当前状态出发最终获胜的概率
- 初始化：赢的状态取 1，输的状态取 0，其他状态取 0.5
- 价值更新（备份更新）：

$$V(S_t) \leftarrow V(S_t) + \alpha [V(S_{t+1}) - V(S_t)]$$

1.6 强化学习算法历史与发展

发展阶段

- 1998 年前：基本理论框架形成
- 1998–2013 年：各种直接策略搜索方法出现
- 2013 年至今：深度强化学习快速发展

代表方向

- 基于值函数的方法：DQN、DNC、AlphaGo
- 基于策略梯度的强化学习
- 基于 EM、基于路径积分、基于回归、基于模型的强化学习

1.7 强化学习的分类

按是否依赖模型

- 基于模型：先学习系统模型，再基于模型求最优策略

- **无模型**：不显式学习环境模型，直接通过交互数据得到最优策略

按策略更新方式

- **基于值函数**：先求最优值函数，再由值函数重构最优策略
- **基于直接策略搜索**：直接在策略空间中搜索最优策略
- **Actor-Critic**：同时逼近值函数和最优策略，类似策略迭代

按回报函数是否已知

- **正向强化学习**：从回报中学习最优策略
- **逆向强化学习**：从专家示例中学习回报函数

按任务形式和规模

分层强化学习、元强化学习、多智能体强化学习、迁移学习。

1.8 强化学习的特点

关键点

核心在于：搜索/探索 + 正确的评估。利用对当前动作的评估进行学习。

2 数学基础

2.1 概率论基础

期望与方差

离散型随机变量期望：

$$\mathbb{E}[X] = \sum_x x P(X = x)$$

连续型随机变量期望：

$$\mathbb{E}[X] = \int x f(x) dx$$

方差：

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

条件概率

在事件 A 已发生条件下，事件 B 发生的概率： $P(B | A)$ 。

随机过程

设 (Ω, Σ, P) 是概率空间，对每个参数 $t \in T$ ， $X(t, \omega)$ 是定义在该概率空间上的随机变量，则随机变量族

$$\{X_t = X(t, \omega); t \in T\}$$

称为该概率空间上的一个随机过程。

2.2 马尔可夫性

关键点

马尔可夫性：系统的下一个状态只与当前状态有关，而与更早的历史无关。

$$P(S_{t+1} | S_t) = P(S_{t+1} | S_1, \dots, S_t)$$

直观理解：当前状态蕴含了所有相关的历史信息；一旦当前状态已知，历史信息就可以被抛弃。

3 多臂赌博机问题

3.1 基本概念

多臂赌博机 (Multi-Armed Bandit):

- 有 K 个臂
- 每次摇动某个臂，会得到服从该臂对应概率分布的回报
- 目标：通过学习知道摇哪个臂能获得更高回报

问题目标：在摇动 N 次的过程中，使总回报最高。

强化学习视角下的表述

- 动作： $a \in \{0, 1, 2, \dots\}$
- 状态：单一状态 s
- 回报： r ，服从不同动作对应的不同回报分布
- 学习目标：在状态 s 下找到最优动作，利用回报 r 学习动作的优劣

3.2 动作价值的定义

动作的真实价值：

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a]$$

动作的估计值（样本平均）：

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}(A_i = a)}{\sum_{i=1}^{t-1} \mathbf{1}(A_i = a)}$$

其中 $\mathbf{1}(A_i = a)$ 表示当第 i 次选择动作 a 时取 1，否则取 0。

3.3 探索与利用

在每个臂都至少尝试一次后，选择下一个臂需要平衡：

- 利用（exploitation）：选择估计值最大的动作

$$a = \arg \max_a q(a)$$

- 探索（exploration）：选择非贪婪动作，以获取更多信息

3.4 增量式更新

关键点

新估计值 = 旧估计值 + 步长 × 预测误差

$$Q_{n+1} = Q_n + \frac{1}{n} (R_n - Q_n)$$

3.5 常见动作选择方法

Upper Confidence Bound (UCB)

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

其中 $Q_t(a)$ 为价值估计， $N_t(a)$ 为动作 a 被选择的次数， c 控制探索强度。

玻尔兹曼分布（Softmax）动作选择

$$\Pr\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}}$$

其中 $H_t(a)$ 表示动作偏好（preference）。

3.6 从多臂赌博机到马尔可夫决策过程

备注

核心区别：

- 多臂赌博机：单状态问题，动作只影响立即回报
- MDP：多状态问题，动作不仅影响回报，还影响状态转移

MDP 中：在状态 s_t 执行动作 a_t ，获得回报 r_t ，转移到下一状态 s_{t+1} ：

$$s_t \xrightarrow{a_t, r_t} s_{t+1}$$

延迟回报：在 MDP 中，动作的好坏不能只看立即回报，还要看未来回报。需要平衡立即回报与将来回报。

Part II

马尔可夫决策过程与动态规划

4 马尔可夫决策过程（MDP）

4.1 定义

关键点

MDP 由五元组描述： (S, A, P, R, γ)

- S ：有限状态集
- A ：有限动作集
- P ：状态转移概率
- R ：回报函数
- γ ：折扣因子

4.2 动力学模型

随机变量 s', r 的联合分布：

$$p(s', r \mid s, a) = \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}$$

状态转移概率

$$p(s' | s, a) = \sum_r p(s', r | s, a)$$

回报的期望

$$R(s, a) = \mathbb{E}[R_t | S_{t-1} = s, A_{t-1} = a] = \sum_r \sum_{s'} r p(s', r | s, a)$$

4.3 状态、动作与回报

状态

状态是任何我们能够知道、并且可能有助于决策的信息。

- DQN 的状态：连续的四帧图像
- AlphaGo 的状态：过去八手的双方棋局

状态表示会严重影响学习效果，目前更像一门艺术。

动作

动作是任何我们希望智能体学习做出的决策，可以是宏观/微观、精神/物质层面动作。

立即回报的设计

立即回报的设置应服务于最终目标：

- 机器人学习走路：回报正比于前进速度
- 机器人逃离迷宫：每走一步给 -1 ，鼓励尽快逃离
- 下棋：赢 $+1$ ，输 -1 ，平局 0

4.4 累积回报与折扣回报

累积回报

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_T$$

幕与任务类型

- 幕 (episode): 一次完整交互序列
- 终止状态 (terminal state): 交互结束的状态
- episodic tasks: 有终止状态的任务
- continuing tasks: 没有终止状态、持续进行的任务

折扣回报

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

其中 $0 \leq \gamma \leq 1$ 。

关键点

折扣率 γ 的含义:

- $\gamma \rightarrow 1$: 智能体更有远见 (farsighted), 重视未来回报
- $\gamma = 0$: 智能体短视 (myopic), 只最大化立即回报
- 若回报有界: $G_t \leq \frac{R_{\max}}{1 - \gamma}$

4.5 策略与值函数

策略的定义

策略 π 是给定状态 s 时在动作集上的一个概率分布:

$$\pi(a | s) = P(A_t = a | S_t = s)$$

常见策略

1. 贪婪策略:

$$\pi(a | s) = \begin{cases} 1, & a = \arg \max_{a \in A} q_*(s, a) \\ 0, & \text{otherwise} \end{cases}$$

2. 随机策略: 按某个概率分布随机选动作

3. 高斯策略: 常用于连续动作空间

4. 玻尔兹曼策略:

$$\pi_{\theta}(a | s) = \frac{\exp(Q(s, a; \theta))}{\sum_b \exp(Q(s, b; \theta))}$$

4.6 值函数与动作价值函数

状态值函数

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

行为值函数

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

备注

由于状态转移概率和策略都可能是随机的，因此折扣累积回报 G_t 本身是随机变量。值函数就是该随机回报的期望。

4.7 贝尔曼方程

值函数与动作价值函数之间的关系

$$v_{\pi}(s) = \sum_{a \in A} \pi(a \mid s) q_{\pi}(s, a)$$

状态值函数的贝尔曼方程

$$v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]$$

行为值函数的贝尔曼方程

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a')]$$

最优贝尔曼方程

$$v^*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v^*(s')]$$

$$q^*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q^*(s', a')]$$

5 动态规划

5.1 动态规划的本质

关键点

动态规划的核心思想：将多阶段决策问题通过贝尔曼方程，转化为多个单阶段决策问题。

离散情形

$$J^*(x(j), j) = \min_{u(j) \in U} \{L(x(j), u(j), j) + J^*(x(j+1), j+1)\}$$

其中 $x(j+1) = f(x(j), u(j), j)$ 。

连续情形与 HJB 方程

$$-\frac{\partial J^*[x(t), t]}{\partial t} = \min_{u(t) \in U} \left\{ L[x(t), u(t), t] + \frac{\partial J^*[x(t), t]}{\partial x^T} f[x(t), u(t), t] \right\}$$

5.2 策略评估 (Policy Evaluation)

输入：策略 π 、状态转移概率 $P_{ss'}^a$ 、回报函数 R_{sa} 、折扣因子 γ

初始化： $V(s) = 0$

迭代更新：

$$v_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(R_{sa} + \gamma \sum_{s' \in S} P_{ss'}^a v_k(s') \right)$$

直到 $v_{k+1} = v_k$ 。该方法属于“期望更新”，一次对所有状态的更新称为“一次状态扫描”。

5.3 策略改进 (Policy Improvement)

基本思想：在每个状态上采用贪婪策略：

$$\pi_{l+1}(s) \in \arg \max_a q_{\pi_l}(s, a)$$

策略改善理论：如果新策略在每个状态上的动作选择都不差于原策略，则新策略不会比原策略差，通常会更好。

贪婪策略计算：

$$q_{\pi_l}(s, a) = R_{sa} + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi_l}(s')$$

$$\pi_{l+1}(s) \in \arg \max_a q_{\pi_l}(s, a)$$

5.4 策略迭代 (Policy Iteration)

关键点

策略迭代由两步交替进行，直到策略不再变化：

1. 策略评估：对当前策略 π_l 求得 V^{π_l}
2. 策略改进： $\pi_{l+1}(s) \in \arg \max_a q_{\pi_l}(s, a)$

终止条件： $\pi_{l+1} = \pi_l$ ，输出 $\pi^* = \pi_l$ 。

关系总结： $V^* \geq V^\pi$ ，当 $\pi = \text{greedy}(V)$ 时，通过反复评估和改进可逐步逼近最优值函数 V^* 和最优策略 π^* 。

5.5 值函数迭代 (Value Iteration)

动机：策略改进一定要等到值函数完全收敛吗？不一定。当 $K = 1$ 时便进行策略改进，可得到值迭代算法。

最优贝尔曼方程：

$$v^*(s) = \max_a \left(R_{sa} + \gamma \sum_{s' \in S} P_{ss'}^a v^*(s') \right)$$

算法：

- 1: 初始化 $v(s) = 0$
- 2: **repeat**
- 3: **for** 每个 s **do**
- 4: $v_{l+1}(s) = \max_a (R_{sa} + \gamma \sum_{s'} P_{ss'}^a v_l(s'))$
- 5: **end for**
- 6: **until** $v_{l+1} = v_l$

收敛后提取策略：

$$\pi(s) = \arg \max_a \left(R_{sa} + \gamma \sum_{s' \in S} P_{ss'}^a v_l(s') \right)$$

小结

特点：将策略评估与策略改进更紧密地结合，不需要每次都把策略评估做到完全收敛，通常比完整策略迭代更直接。

Part III

无模型方法

6 蒙特卡洛方法

6.1 基本思想

关键点

蒙特卡洛方法的核心思想：利用经验平均代替随机变量的期望。

一次实验（episode）是从初始状态开始，到终止状态结束的一条完整轨迹。对某个时刻 t 之后的折扣回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$$

6.2 与动态规划策略评估对比

- **DP**：需要已知模型，每次进行一次状态扫描
- **MC**：每次进行一次完整试验，用试验中观察到的回报估计值函数

6.3 First-visit MC 与 Every-visit MC

- **First-visit MC**：每幕中只使用某状态第一次出现后的回报进行估计
- **Every-visit MC**：每幕中该状态每次出现后的回报都用于估计

根据大数定律： $\hat{v}(s) \rightarrow v_\pi(s)$ ，当 $N(s) \rightarrow \infty$ 。

6.4 探索型初始化

为保证充分访问，采用探索型初始化（exploring starts）：每个状态都有一定几率作为初始状态；更强的形式是每个状态-行为对都有机会作为初始状态-动作对出现。

6.5 蒙特卡洛评估行为值函数

在模型未知时，最重要的是评估行为值函数：

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

备注

蒙特卡洛评估与动态规划评估的区别：

- MC 不需要 bootstrap
- MC 对单个状态的估计相对独立
- MC 对单个状态的估计复杂度不直接依赖于整个状态空间大小
- DP 依赖模型；MC 依赖采样得到的完整 episode

6.6 蒙特卡洛策略改进

贪婪改进形式：

$$\pi(s) = \arg \max_a q_{\pi}(s, a)$$

蒙特卡洛强化学习：探索初始化

- 1: 初始化 $Q(s, a)$ 、 $\pi(s)$ 、 $\text{Returns}(s, a)$
- 2: **repeat**
- 3: 随机选择初始状态-动作对 (S_0, A_0)
- 4: 从 (S_0, A_0) 出发，按策略 π 生成一个 episode
- 5: **for** episode 中每个 (s, a) **do**
- 6: 计算 G （第一次出现后的回报）
- 7: 将 G 附加到 $\text{Returns}(s, a)$
- 8: $Q(s, a) \leftarrow \text{average}(\text{Returns}(s, a))$
- 9: **end for**
- 10: $\pi(s) \leftarrow \arg \max_a Q(s, a)$
- 11: **until** 收敛

6.7 On-policy 与 Off-policy

基本要求

策略必须是永久温和的，即对任意状态 s 和动作 a ： $\pi(a|s) > 0$ 。

On-policy MC

产生数据的策略、被评估的策略、被改进的策略三者是同一个策略。典型做法是使用 ε -soft 策略：

$$\pi(a|s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A(s)|}, & a = a^* \\ \frac{\varepsilon}{|A(s)|}, & a \neq a^* \end{cases}$$

Off-policy MC

使用两个不同的策略：

- 目标策略 π ：要学习和改进的策略，通常较贪婪
- 行为策略 μ ：用来实际采样，负责探索

覆盖性条件：若 $\pi(a|s) > 0$ ，则必须有 $\mu(a|s) > 0$ 。

6.8 重要性采样

普通重要性采样

$$\mathbb{E}[f] \approx \frac{1}{N} \sum_{n=1}^N \omega_n f(z_n), \quad \omega_n = \frac{p(z_n)}{q(z_n)}$$

特点：无偏估计，但方差可能非常大。

加权重要性采样

$$\mathbb{E}[f] \approx \frac{\sum_{n=1}^N \omega_n f(z_n)}{\sum_{n=1}^N \omega_n}$$

特点：一般有偏，但方差更小、更稳定。

MC 中的重要性采样

重要性比率只与动作概率有关：

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}$$

普通重要性采样估计：

$$V(s) = \frac{\sum_{t \in T(s)} \rho_t G_t}{|T(s)|}$$

加权重要性采样估计：

$$V(s) = \frac{\sum_{t \in T(s)} \rho_t G_t}{\sum_{t \in T(s)} \rho_t}$$

7 时间差分学习 (TD)

7.1 TD 学习的基本思想

关键点

- TD 学习是采样更新
- DP 学习是期望更新

DP、MC、TD 的对比

- **MC**: 利用完整回报 G_t 估计值函数（采样平均逼近期望）
- **TD**: 结合 MC 和 DP，采样得到目标，并利用当前估计值 $V(S_{t+1})$ 进行自举
- **DP**: 依赖环境模型提供期望，也利用当前估计值进行更新

7.2 增量式 MC 与 TD(0)

增量式 MC:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

TD(0):

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

其中 $R_{t+1} + \gamma V(S_{t+1})$ 称为 **TD 目标**。

关键点

偏差-方差权衡:

- G_t 是 $v_\pi(S_t)$ 的无偏估计
- 真实 TD 目标 $R_{t+1} + \gamma v_\pi(S_{t+1})$ 也是无偏估计
- 实际使用的 TD 目标 $R_{t+1} + \gamma V(S_{t+1})$ 是有偏估计
- TD 目标的方差通常比 MC 的回报 G_t 小很多

7.3 时间差分学习的优势

- TD 不需要环境模型
- 与 MC 相比，TD 只需要等待一个时间步
- TD 只评估当前动作，与后继动作序列无直接关系
- TD 方法的收敛性已经得到证明

7.4 表格型强化学习算法

Sarsa: On-Policy TD

学习行为值函数 $Q(s, a)$ ，基本数据单元是五元组 (s, a, r, s', a') 。行动策略和评估策略都是同一个 ϵ -贪婪策略。

更新公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Q-learning: Off-policy TD

直接逼近最优行为值函数。行动策略通常为 ϵ -贪婪，目标策略为贪婪。

更新公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Expected Sarsa

把下一状态下动作值的随机采样，替换成对策略分布下的期望，消除部分方差。

Double Q-learning

问题：Q-learning 中的 $\max_{a'} Q(s', a')$ 会因最优化操作导致最大化偏差。

思想：使用两个值函数逼近器，一个负责选择动作，另一个负责评估该动作的值，减轻过高估计问题。

7.5 ϵ -贪婪策略

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|}, & a = \arg \max_a Q(s, a) \\ \frac{\epsilon}{|A(s)|}, & \text{其他动作} \end{cases}$$

Part IV

函数逼近与深度强化学习

8 函数逼近方法

8.1 表格表示的局限

行为值函数总数与状态动作空间大小成正比： $q(s, a)$, $s \in S, a \in A$ 。

关键点

问题：当状态空间连续或状态数无穷多时，存储空间趋于无穷大，学习速度很慢。例如图像状态 (400, 500) 分辨率，像素维度达 200000。

8.2 函数逼近表示

利用函数逼近方法估计值函数： $\hat{q}(s, a; \theta)$ 。

逼近方法分类：

- 参数化逼近：线性参数化（人工设计基底）、非线性参数化（神经网络、决策树、模糊网络）
- 非参数化逼近：基于核的方法、基于高斯回归的方法

备注

与普通监督学习相比，强化学习中的函数逼近存在以下特点：

- 非静态性
- 自举 (bootstrapping)
- 延迟目标

8.3 值函数估计过程

表格型值函数估计中的 Backup 值

DP 的行为值函数 Bellman 期望形式：

$$Q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \sum_{a' \in A} \pi(a'|s') Q_{\pi}(s', a')$$

此外还有：MC 的回溯值（完整回报）、TD 的回溯值（一步 TD 目标）。

函数逼近视角下的值函数估计

用函数逼近器表示值函数，训练目标来自 Backup 值。

8.4 随机梯度下降

蒙特卡罗方法下的值函数估计

策略评估过程可视为最小化预测值与回报之间误差的过程。

半梯度下降

对于 TD(0)、DP 等方法，目标依赖于当前参数估计，因此常采用半梯度方法。

TD 目标：

$$U_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, a; \theta_t)$$

关键点

半梯度方法：更新时只考虑参数 θ 对估计值函数的影响，忽略其对目标函数 U_t 的影响。好处：学习速度快，可应用于连续系统。

8.5 值函数的线性逼近

关键点

线性逼近的优点：线性情况下只有一个最优值，因此能收敛到全局最优。

常用基函数类型：多项式基函数、傅里叶基函数、径向基函数。

8.6 批方法

增量式方法：计算简单，但样本效率通常不高。

批方法：给定经验数据集，直接寻找整体上最好的拟合函数值。

最小二乘方法

最小二乘蒙特卡罗方法 (LSMC)：

$$\theta = \left(\sum_t \phi(s_t) \phi(s_t)^T \right)^{-1} \left(\sum_t \phi(s_t) G_t \right)$$

最小二乘时间差分方法 (LSTD)：

$$\theta = \left(\sum_t \phi(s_t) (\phi(s_t) - \gamma \phi(s_{t+1}))^T \right)^{-1} \left(\sum_t \phi(s_t) R_{t+1} \right)$$

最小二乘 TD(λ) 方法:

$$\theta = \left(\sum_t E_t (\phi(s_t) - \gamma \phi(s_{t+1}))^T \right)^{-1} \left(\sum_t E_t R_{t+1} \right)$$

备注

表格型强化学习是函数逼近的特殊情形: $Q(s, a) = \phi(s, a)^T \Theta$, 参数 Θ 对应于原表格表示中的行为值函数项。

9 神经网络与深度学习框架

9.1 前向神经网络

神经网络可以看作是基函数参数化的一种方法。前向神经网络包含多层结构, 通过逐层前向计算得到输出, 输出层给出最终预测结果, 隐层可视为参数化的基函数。

9.2 后向传播

目标: 给定训练样本, 定义损失函数, 利用梯度下降法更新权值和偏置。

基本步骤:

1. 正向计算每层输出
2. 计算最后一层残差
3. 残差从输出层向前逐层传播
4. 计算各层参数的偏导数, 并完成更新

9.3 卷积神经网络 (CNN)

关键点

核心特点:

- 卷积: 权值共享、稀疏连接
- 池化: 提取抽象特征、进一步减少参数量

典型结构: LeNet 网络。

10 DQN 及其变种

10.1 DQN 介绍

关键点

DQN 的核心思想：

- 框架基于 Q-learning
- 值函数 $Q(s, a)$ 由卷积神经网络表示

DQN 的关键技巧：

1. 用卷积神经网络逼近行为值函数
2. 使用经验回放（**Experience Replay**）训练强化学习过程
3. 设置目标网络（**Target Network**），处理时间差分学习中的目标不稳定问题

10.2 DQN 变种

Double DQN

DQN 使用目标网络处理 TD 目标的非稳定性。Double DQN 进一步针对 max 操作引入的过优化问题，采用两个值函数网络：一个用于选择动作，一个用于评估动作值。

Prioritized DQN

经验回放从均匀采样变为优先采样。

Dueling Network

改进网络结构，以更好地分解状态价值与动作优势。

DRQN (Deep Recurrent Q-Network)

面向部分可观测马尔可夫决策过程（POMDP），在 Q-learning 结构中加入循环网络处理历史信息。

Rainbow

将多种 DQN 改进统一组合起来，形成更强的深度强化学习算法。

Part V

策略梯度方法

11 策略搜索方法

11.1 为什么要策略搜索

关键点

关键问题：动作的选择一定要依赖值函数，尤其是行为值函数 $Q(s, a)$ 吗？不一定。

当用值函数进行策略改进时，通常需要解 $\arg \max_a Q(s, a)$ 。但如果动作空间很大，或者动作空间是连续的，这个优化往往难以直接求解。因此，可以选择**直接学习策略**。

11.2 策略搜索方法分类

无模型的策略搜索方法

- 策略梯度方法
- TRPO（单调步长、方差约简）
- 统计学习方法、路径积分方法

按策略形式分类

- 随机策略： $\pi_{\theta}(a | s)$
- 确定性策略： $a = \mu_{\theta}(s)$ ，典型方法 DDPG

基于模型的策略搜索方法

指导策略搜索（GPS）、传统控制器 + 监督学习。

11.3 策略搜索的好处

- 适用于离散动作空间的参数化策略，可逼近确定性策略
- 可逼近任意概率分布
- 策略本身往往比值函数更容易逼近（如 PID 控制）
- 更容易加入先验知识（控制结构、物理知识、经验规律）

12 随机策略梯度

12.1 强化学习的目标函数

关键点

强化学习的目标是最大化策略 π 下的期望折扣回报：

$$J(\pi) = \mathbb{E} \left[\sum_{k=t}^{\infty} \gamma^{k-t} r(s_k, a_k) \right]$$

折扣状态分布：

$$\rho^{\pi}(s') = \int_S \sum_{t=1}^{\infty} \gamma^{t-1} p_1(s) p(s' | s, t, \pi) ds$$

目标也可写为：

$$J(\pi_{\theta}) = \int_S \rho^{\pi_{\theta}}(s) \int_A \pi_{\theta}(a | s) r(s, a) da ds$$

12.2 策略梯度定理

关键点

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi_{\theta}}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a)]$$

备注

关键结论： 尽管状态分布 $\rho^{\pi_{\theta}}(s)$ 依赖于参数 θ ，但策略梯度表达式本身不需要对状态分布求导。难点转化为：如何估计行为值函数 $Q^{\pi}(s, a)$ 。

12.3 似然率策略梯度推导

轨迹概率分解

$$p_{\theta}(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

对数轨迹似然：

$$\log p_{\theta}(\tau) = \log p(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) + \sum_{t=0}^{T-1} \log P(s_{t+1} | s_t, a_t)$$

由于环境动力学 $P(s_{t+1} | s_t, a_t)$ 与策略参数 θ 无关，求导后只剩策略项。

经验平均估计

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(\tau^{(i)}) \nabla_{\theta} \log p_{\theta}(\tau^{(i)})$$

关键点

直观理解：增加高回报路径的概率，减小低回报路径的概率。策略梯度是在”奖励好的轨迹、抑制差的轨迹”。

12.4 REINFORCE: Monte Carlo Policy Gradient

关键点

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

特点：无偏、易实现，但方差较大。

12.5 减小方差的方法

1. 引入基线 (Baseline)

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b)]$$

性质：仍然无偏，合理选取 b 可以显著降低方差。

2. 修改值函数的使用方式

两个重要事实：

- 当前动作与过去回报无关
- 当前回报只与过去动作有关

因此不必把整条轨迹的总回报都分配给每一步动作，可以只使用”未来回报”部分。这是从总回报 $R(\tau)$ 转向分时刻回报 G_t 、再引入优势函数的基础。

12.6 Actor-Critic 框架

- **Actor:** 调整策略网络参数 θ
- **Critic:** 调整行为值函数网络参数 w ，用 $Q_w(s, a)$ 逼近 $Q^{\pi}(s, a)$

相容函数

在随机策略梯度中，一种相容形式为：

$$Q_w(s, a) = \nabla_{\theta} \log \pi_{\theta}(a | s)^T w$$

参数 w 应使均方误差最小：

$$\varepsilon(w) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} \left[(Q^{\pi}(s, a) - Q_w(s, a))^2 \right]$$

12.7 随机 Off-policy Actor-Critic

采样策略为 β ，目标策略为 π ：

$$J_{\beta}(\pi_{\theta}) = \int_S \rho^{\beta}(s) V^{\pi}(s) ds = \int_S \rho^{\beta}(s) \int_A \pi_{\theta}(a | s) Q^{\pi}(s, a) da ds$$

13 基于置信域的方法：TRPO 与 PPO

13.1 策略梯度算法的缺点

关键点

- 步长非常重要，很难在整个优化过程中选择合适的学习步长
- 策略经常会过早收敛到次优的、几乎确定性的策略
- 当步长不合适时，学到的策略可能很差；由坏策略采集到的数据也不好，学习过程可能崩溃

13.2 替代回报函数

基本思想：策略梯度算法直接优化原始回报函数，但原目标难以稳定优化，因此寻找一个替代回报函数来优化。

优势函数

$$A^{\pi_{\text{old}}}(s, a) = Q^{\pi_{\text{old}}}(s, a) - V^{\pi_{\text{old}}}(s)$$

等价替代函数

$$L_{\pi_{\text{old}}}(\pi) = \eta(\pi_{\text{old}}) + \sum_s \rho_{\pi_{\text{old}}}(s) \sum_a \pi(a | s) A^{\pi_{\text{old}}}(s, a)$$

两个关键技巧

1. 对状态分布的处理：忽略状态分布变化，仍使用旧策略对应的状态分布
2. 对动作分布的处理：使用重要性采样修正新旧策略的差异

重要性采样形式

$$L_{\theta_{\text{old}}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right]$$

13.3 理论下界

关键点

定理 1：令 $\alpha = D_{TV}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})$ ，则

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - \frac{4\epsilon\gamma}{(1-\gamma)^2} \alpha^2$$

其中 $\epsilon = \max_{s,a} |A^{\pi_{\text{old}}}(s,a)|$ 。

含义：若新旧策略差异过大，替代目标对真实回报的近似会变差。

13.4 TRPO 实用算法

问题形式化：

$$\max_{\theta} L_{\theta_{\text{old}}}(\theta) \quad \text{s.t.} \quad \bar{D}_{KL}^{\rho_{\theta_{\text{old}}}}(\theta_{\text{old}}, \theta) \leq \delta$$

四个关键技巧

1. 忽略状态分布变化
2. 使用重要性采样
3. 利用平均 **KL 散度** 替代逐状态约束
4. 利用当前策略对应的状态分布

共轭梯度法搜索可行方向

将目标函数线性化、将约束函数二次近似后，优化问题转化为局部二次规划。

Fisher 信息矩阵：

$$A_{ij} = \frac{\partial}{\partial \theta_i} \frac{\partial}{\partial \theta_j} D_{KL}(\theta_{\text{old}}, \theta)$$

搜索方向：近似求解 $Ax = g$ ，得到自然梯度方向 $x = A^{-1}g$ 。

线性搜索：保证 KL 约束满足、保证替代目标确实提升、避免更新过大导致性能退化。

13.5 近端策略优化算法（PPO）

CLIP 目标函数

关键点

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

$$\text{其中 } r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}。$$

含义：

- 当 $\hat{A}_t > 0$ 时，若 r_t 超过 $1 + \epsilon$ ，则限制其继续增大
- 当 $\hat{A}_t < 0$ 时，若 r_t 小于 $1 - \epsilon$ ，则限制其继续减小
- 本质：优化一个更低的界，防止策略更新过猛

PPO 总目标函数

$$L(\theta) = \hat{\mathbb{E}}_t \left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right]$$

其中 L^{VF} 为值函数损失， S 为策略熵， c_1, c_2 为权重系数。

优势函数估计（GAE）

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t-1}\delta_{T-1}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

13.6 回报函数对比

小结

策略梯度：

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$$

TRPO：

$$L^{TRPO}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right]$$

并加入 KL 约束。

PPO：

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

13.7 TRPO 用到的信息论

熵

离散系统熵： $H(X) = - \sum_x p(x) \log p(x)$

连续系统熵： $H(X) = - \int p(x) \log p(x) dx$

交叉熵

$$H(p, q) = - \sum_x p(x) \log q(x)$$

KL 散度

$$D_{KL}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

在 TRPO 中，用它来约束新旧策略之间的差异，控制更新步长。

Part VI

确定性策略与多智能体

14 确定性策略梯度

14.1 随机策略与确定性策略

关键点

- 随机策略: $a \sim \pi_\theta(a | s)$, 需要对状态空间和动作空间积分
- 确定性策略: $a = \mu_\theta(s)$, 只需对状态空间积分

当动作空间维数较高时, 确定性策略方法更有优势。

随机策略的参数化

随机策略通常可写为确定性策略 + 随机扰动:

$$a = \mu_\theta(s) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

即 $a \sim \mathcal{N}(\mu_\theta(s), \sigma^2)$ 。

以线性策略为例: $\mu_\theta(s) = \theta^T \phi(s)$, 其中 $\phi(s)$ 为状态特征, θ 为参数, 方差 σ 控制探索性。

14.2 确定性值函数

行为值函数:

$$Q^\pi(s_t, a_t) = \mathbb{E} \left[\sum_{i=t}^{\infty} \gamma^{i-t} r_i \mid s_t, a_t \right]$$

迭代形式:

$$Q^\pi(s_t, a_t) = \mathbb{E} [r(s_t, a_t) + \gamma Q^\pi(s_{t+1}, a_{t+1})]$$

若目标策略为确定性策略 μ :

$$Q^\mu(s_t, a_t) = \mathbb{E} [r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))]$$

14.3 确定性策略梯度

关键点

策略改善的思路是在策略梯度方向上更新参数。根据链式法则, 确定性策略梯度定理的核心思想:

- 动作由确定性策略 $\mu_\theta(s)$ 直接给出
- 梯度通过 $Q^\mu(s, a)$ 对动作的导数, 以及动作对参数 θ 的导数传递

14.4 相容函数逼近

条件 1: 动作梯度匹配

$$\nabla_a Q_w(s, a)|_{a=\mu_\theta(s)} = \nabla_\theta \mu_\theta(s)^T w$$

条件 2: 最小化均方误差

$$\text{MSE}(w) = \mathbb{E} [\varepsilon(s; \theta, w)^T \varepsilon(s; \theta, w)]$$

其中 $\varepsilon(s; \theta, w) = \nabla_a Q_w(s, a)|_{a=\mu_\theta(s)} - \nabla_a Q^\mu(s, a)|_{a=\mu_\theta(s)}$ 。

满足条件 1 的相容函数形式

$$Q_w(s, a) = (a - \mu_\theta(s))^T \nabla_\theta \mu_\theta(s)^T w + V^\mu(s)$$

14.5 DDPG

使用神经网络逼近时面临的问题

- 神经网络训练通常假设数据独立同分布，而强化学习中顺序采集的数据存在明显相关性
- 若能采用 mini-batch 优化，可更充分利用数据

DQN 的成功经验

经验回放 (Experience Replay)、独立目标网络 (Target Network)。

关键点

DDPG 的核心机制:

- 行动策略为随机策略
- 使用目标网络
- 使用经验回放
- 目标网络参数进行延迟/软更新
- 采用 Actor-Critic 结构处理连续动作问题

14.6 TD3: DDPG 的扩展

DDPG 存在的问题

- 值函数过高估计
- 值函数方差较高
- 策略网络与值函数网络之间存在耦合关系

解决思路

- 使用独立目标网络
- **Twin Delayed (双延迟) 机制**: 目标网络更新延迟、策略更新延迟、目标策略平滑 (Target Policy Smoothing)

14.7 D4PG: 分布式分布式确定性策略梯度

备注

全称: Distributed Distributional Deep Deterministic Policy Gradient

核心思想: 任何改善 critic 学习质量的方法, 都会直接改善 actor 更新质量。值分布建模了函数逼近等带来的随机性, 利用值分布更新 critic 可以直接提升算法表现。

D4PG 使用的关键技巧

- 分布式值函数强化学习, 得到更细致的 critic
- 分布式并行 Actor
- N -step 回报
- 优先经验回放采样

15 博弈论与多智能体强化学习

15.1 多智能体系统与分类

多智能体强化学习分类 (按任务类型)

- 完全协作多智能体系统
- 完全竞争多智能体系统
- 混合策略多智能体系统

多智能体强化学习的挑战

关键点

- MDP 假设环境是静态的；但从单智能体视角看，其他智能体本身是动态环境，因此环境不再平稳
- 当智能体之间是竞争关系时，最优策略往往不存在，需要求解均衡解

15.2 博弈论基本分类

按参与人行动先后顺序

- 静态博弈：参与人同时选择行动（如猜丁壳），常用标准型表述（矩阵博弈）
- 动态博弈：参与人的动作有先后顺序，且后行动者可观察前行动者的选择（如围棋），常用扩展型表述（博弈树）

按参与人对信息掌握程度

- 完美信息博弈：参与人对相关信息已知（如棋类游戏）
- 不完美信息博弈：参与人对相关信息并不完全知道（如牌类游戏）

15.3 完美信息静态博弈的基本概念

基本要素

- 参与者：参与博弈的智能体 i
- 动作空间： A_i ，联合动作 $a = (a_1, a_2, \dots, a_n)$
- 策略： σ_i
 - 纯策略：选择某一行为的概率为 1
 - 混合策略：选择某一行为的概率小于 1
- 联合策略： $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_n)$
- 回报函数： $R_i(a)$
- 值函数： $V_i(\sigma)$

15.4 最优响应与纳什均衡

最优响应

当其他玩家策略固定时，玩家 k 的最优响应：

$$R_k(\sigma_{-k}) = \{\sigma'_k \in A_k \mid \mu_k(\sigma'_k, \sigma_{-k}) \geq \mu_k(\sigma_k, \sigma_{-k}), \forall \sigma_k \in A_k\}$$

纳什均衡

关键点

当所有玩家都采取各自的最优响应时，称该联合策略为纳什均衡。

$$V_i(\sigma_i^*, \sigma_{-i}^*) \geq V_i(\sigma_i, \sigma_{-i}^*), \quad \forall i, \forall \sigma_i$$

例子：囚徒困境中，(坦白, 坦白) 是纳什均衡策略。

15.5 双人零和博弈

双人联合行为对应的回报构成矩阵，因此称为矩阵博弈。

关键点

求解双人零和矩阵博弈中的纳什均衡，等价于求解相应的极小极大问题，进一步可转化为线性规划问题，使用单纯形法求解。

15.6 非完美信息博弈：扩展式博弈

七元组描述： $(H, Z, P, p, u, I, \sigma)$

- H ：历史节点集合
- Z ：终止状态集合
- P ：玩家集合
- p ：非终止状态到玩家的映射
- u ：终止状态到实数的映射（回报）
- I ：信息集，其中所有节点对玩家 i 来说不可区分
- σ ：策略

备注

perfect recall（完美回放）：指玩家对于先前已知的信息不会发生任何程度的遗忘。

15.7 随机博弈（马尔科夫博弈）

随机博弈（Stochastic game）核心元素： n （玩家个数）、联合状态转移概率、联合立即回报、联合动作空间、局部策略、联合策略。

值函数特征

在马尔科夫博弈中，每个智能体的行为值函数依赖于联合动作。

15.8 多智能体强化学习

回报定义

对于智能体 i ：

$$V_i^\pi(s) = \mathbb{E}_\pi \left\{ \sum_{t=0}^{\infty} \gamma^t r_i(t+1) \mid s(0) = s \right\}$$

其中 $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ 为联合策略。

关键点

注意：一般情况下，不可能同时最大化每个智能体的目标函数，因为不同智能体的目标可能相互矛盾。最优响应与纳什均衡的概念可扩展到马尔科夫博弈中。

15.9 多智能体 Q-learning

更新形式

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha [R + \gamma V(s')]$$

下一个状态值函数 $V(s')$ 的获取方式

1. 对手建模方法：统计其他智能体可能联合动作的频率 $F(s, a_{-i})$ ：

$$V_i(s) = \max_{a_i \in A_i} \sum_{a_{-i} \in A_{-i}} F(s, a_{-i}) Q_i(s, a_i, a_{-i})$$

2. 假设对手按固定规则选动作（如零和博弈中最小最大原则）：

$$V(s) = \min_{\mu} \max_{\sigma} \sum_{a \in A} \sigma(a) \mu(a') Q(s, a, a')$$

3. 假设下一个状态执行均衡策略（Nash-Q、相关均衡、Stackelberg 均衡）：

$$V(s) = \text{Nash}_s Q(s, \cdot)$$

15.10 多智能体强化学习算法类型

1. 完全协作强化学习算法

示例：Cooperative and Distributed Reinforcement Learning of Drones for Field Coverage。

2. 完全竞争：Minimax Q-learning

定义值函数时采用最小最大思想。若相关博弈模型已知，可利用线性规划方法求解。

3. 混合任务：Nash-Q 学习

- 利用二次规划更新行为值函数
- 通过求解均衡策略进行更新
- 收敛条件：每个状态处的博弈都有一个全局最优点或者鞍点（过于严格，实际中很难满足）

4. 混合任务：朋友或敌人 Q-learning

其他玩家要么被视为”朋友”（共同最大化），要么被视为”敌人”（共同最小化）。 N 玩家一般和随机博弈可被看作一个动作集扩展的两玩家零和博弈。

15.11 WoLF 策略爬山算法

关键点

基本思想：WoLF-PHC 算法仅需要每个玩家自己的动作信息，可显著约减空间复杂度。组合了两个方法：

- **Win or Learn Fast (WoLF)：**做得比期望差时快速学习，做得好时谨慎学习
- **Policy Hill-Climbing (PHC)：**策略爬山算法，满足合理性要求

特点：策略爬山算法仅能保证在静态环境下，单智能体收敛到最优策略。

15.12 基于微分对策的方法

内容要点：运动学方程、性能指标函数、定义双人零和微分对策、推导 Hamilton-Jacobi-Isaacs 方程。

15.13 深度强化学习算法：MADDPG

MADDPG 算法框架、MADDPG 算法伪代码、论文讲解。

附录：Python 与 NumPy 基础

Python 基础

`print()` 语句常见格式化输出：字符串 `%s`、整数 `%d`、浮点数 `%f`。

条件语句 `if...elif...else`；循环语句 `for...in`、`while`，配合 `continue` 和 `break`。

函数定义使用 `def`；类与面向对象使用 `class`，成员变量用 `self`。

NumPy 基础

创建矩阵：`np.array([[1,2,3],[2,4,6]])`

NumPy 矩阵属性：维数 `ndim`、形状 `shape`、大小 `size`。

常用操作：

- 升维：`B[np.newaxis, :]`
- 全零矩阵：`np.zeros((3,3))`
- 整数数组：`np.arange(10)`
- 改变形状：`D.reshape(2,5)`
- 矩阵乘法：`np.dot(A,b)`；逐元素乘法：`A*B`
- 转置：`np.transpose(A)`
- 行合并：`np.hstack`；列合并：`np.vstack`

参考材料

- Sutton, Barto: *Reinforcement Learning: An Introduction*
- 郭宪、方勇纯：《深入浅出强化学习：原理入门》
- 郭宪、宋俊潇、方勇纯：《深入浅出强化学习：编程实战》
- 各类公开发表的强化学习论文